

RAG Document Assistant — Q&A over a PDF

Applied AI / GenAI · Adewale Adeyinka Falade · LangChain, OpenAI (GPT-4o), ChromaDB

What It Does

Instead of reading a long document to find an answer, the user asks a question in plain English. The system retrieves the most relevant passages and feeds them to the model as context, so answers are grounded in the source rather than the model's general memory — reducing hallucination and keeping responses on-topic.

How It Works

1. **Load** — PDF loaded with PyPDFLoader.
2. **Chunk** — RecursiveCharacterTextSplitter (size 1000, overlap 200) keeps passages coherent.
3. **Embed** — each chunk converted to a vector with OpenAI embeddings.
4. **Store** — vectors indexed in a ChromaDB vector store.
5. **Retrieve** — top 3 most similar chunks pulled per question by semantic similarity.
6. **Generate** — retrieved context + question sent to GPT-4o for a grounded answer.

What It Demonstrates

- End-to-end RAG with LangChain: load, chunk, embed, store, retrieve, generate.
- Semantic search and context construction over a real document.
- Grounded, verifiable answers drawn only from the indexed source text.

API keys removed; set your own OPENAI_API_KEY to run. The demo document is copyrighted and not included — point the loader at any PDF to reproduce the pipeline.